

Airbnb Web App

SDE Fullstack Assignment

Description

Build a functional clone of the Airbnb web application that replicates Airbnb's design, user experience, and core booking workflows.

The platform should allow users to browse and search property listings, view listing details, filter by criteria, book stays for a date range, and (as a host) create and manage their own listings, all within the clean, photo-forward interface of the original Airbnb app.

Your implementation should visually and functionally feel like a modern Airbnb marketplace. The focus is on recreating the Airbnb experience and core browse/search/booking workflows; real payments can be mocked.

Note: All data, images, media, and external services can be mocked, you do not need to integrate with any real APIs. Use placeholder or hardcoded data wherever needed to demonstrate the intended functionality.

Note: Your application should match the original application in terms of UI and UX, look and feel should be exactly the same.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)

Note: real payment processing is out of scope, a mocked checkout is sufficient.

Map/geolocation can be a static map image or a lightweight map library.

Core Features (Must Have)

1. Home & Search

Recreate the Airbnb home/explore view.

- Grid of listing cards with photo, title, location, price/night, and rating
- Search bar (location + date range + guests)
- Category / filter row (price range, property type, amenities, etc.)

- Pagination or infinite scroll

2. Listing Detail Page

- Photo gallery
- Title, description, location, amenities, host info
- Availability calendar / date-range picker
- Price breakdown (nightly rate × nights + fees)
- Reviews section

3. Booking Flow

Implement end-to-end booking.

- Select date range and guest count with validation (no overlapping/unavailable dates)
- Booking summary and mocked checkout/confirmation
- A “My Trips” view listing the user's bookings
- All bookings must persist and block those dates on the listing

4. Host Experience (CRUD)

Implement full CRUD for listings as a host.

- Create a listing (title, description, photos via URL/upload, price, location, amenities)
- Edit and delete listings
- A host dashboard of owned listings and their bookings
- All listing data must persist

5. Airbnb Experience

The application should closely resemble the Airbnb experience, including:

- Navigation and layout (explore grid + detail view)
- Cards, galleries, date pickers, and modals
- Search, filters, and pagination
- Notifications / toasts
- Wishlist / favorites (can be simple)

The goal is to make the application feel like Airbnb rather than a generic listings CRUD app.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Real payment processing (mock the checkout)
- Messaging between guest and host
- Real-time map with live pricing pins (a static/basic map is fine)
- Identity verification
- Real user authentication may be simplified/mocked, but a notion of “guest vs host” is needed

Bonus (Optional)

- Interactive map with listing pins
- Leave a review after a completed stay
- Superhost badges / ratings aggregation
- Image upload to cloud storage
- Dark mode
- Responsive design (mobile, tablet, desktop)

Important Notes

- UI Design: your application should totally resemble Airbnb's design. Study Airbnb's UI carefully before starting.
- Sample Data: seed your database. Seed a variety of listings (with photos), a few users acting as hosts, and some existing bookings so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including search, availability, and the booking flow
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships
Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components

Code Understanding	Ability to explain your code during evaluation
---------------------------	--

Timeline

Estimated effort: approximately 24 hours of work.

Submission Deadline: as communicated alongside this assignment.

Good luck!